Forward Deployed Engineer (FDE)
Top-Down Learning Roadmap

Goal: learn by building one real application and progressively turning it into a production-style, customer-facing system.

Core Philosophy

Do not learn every concept first. Build the system, encounter a problem, learn the technology needed to solve it, and apply it immediately.

Application → Deployment → Kubernetes → Observability → AI/Agents → Customer Problem → Production

Project: Task Management Platform

Layer	Suggested Technology	Purpose
Frontend	React + TypeScript	User interface
Backend	FastAPI + Python	REST API / business logic
Database	PostgreSQL	Persistent application data
Cache / Jobs	Redis	Caching and background work
Containers	Docker / Docker Compose	Local reproducible environment
CI/CD	GitHub Actions	Automated build and deployment
Cloud	AWS / Azure / GCP	Real deployment
Orchestration	Kubernetes	Production-style deployment
Observability	Prometheus + Grafana + OpenTelemetry	Metrics, dashboards, traces
AI	LLM + tool-calling agent	Natural-language task operations

Progressive Build Stages

1. Build the MVP

Learn: React → FastAPI → PostgreSQL
Deliver: Create/login users, create/update/delete tasks, assignment, priority, status, due dates, search/filter.

2. Containerize

Learn: Docker + Compose
Deliver: Run frontend, backend, database, and Redis consistently with one command.

3. Deploy

Learn: Container Registry + Cloud + CI/CD
Deliver: Push code → build image → publish image → deploy automatically.

4. Kubernetes

Learn: Deployments, Services, Ingress, ConfigMaps, Secrets, PVs
Deliver: Move the same application to Kubernetes and practice scaling, rollouts, failures, and recovery.

5. Observability

Learn: Logs + Metrics + Traces
Deliver: Answer: What is broken? Where? Why? Add dashboards and alerts.

6. AI Agent

Learn: LLM + tools + APIs
Deliver: Let users create, search, summarize, and manage tasks using natural language.

7. Customer Scenario

Learn: Fictional customer + requirements
Deliver: Treat the system as a customer deployment: gather requirements, design, implement, deploy, support, iterate.

8. Production Hardening

Learn: Security, reliability, performance, documentation
Deliver: Add auth controls, secrets management, backups, health checks, resource limits, testing, and runbooks.

FDE Learning Loop

Customer problem

↓

Understand requirements

↓

Design a solution

↓

Build

↓

Deploy

↓

Observe

↓

Customer uses it

↓

Something breaks / feedback arrives

↓

Debug, improve, document

↓

Repeat

First Milestone — Start Here

■ Create a GitHub repository for the Task Management Platform.

■ Build a basic React frontend.

■ Build a FastAPI backend.

■ Connect the backend to PostgreSQL.

■ Implement user authentication.

■ Implement CRUD operations for tasks.

■ Make the frontend consume the backend API.

■ Run the complete system locally.

After MVP: Ask These Questions

• How do I package this application so someone else can run it? → Docker

• How do I deploy it automatically? → CI/CD

• How do I run it reliably in the cloud? → Kubernetes

• How do I know why it is slow or broken? → Observability

• How can users interact with it using natural language? → AI agent

• How would I deploy this for a real customer? → FDE workflow

End Goal

By the end, you should be able to demonstrate one evolving system rather than a collection of disconnected tutorials: **idea → application → Docker → CI/CD → cloud → Kubernetes → observability → AI agent → customer deployment → production support.**

Important: the technologies are secondary. The main skill being developed is the ability to take an ambiguous customer problem and turn it into a working, deployed, observable, maintainable solution.